

Web Applications



Tauhid Zaman

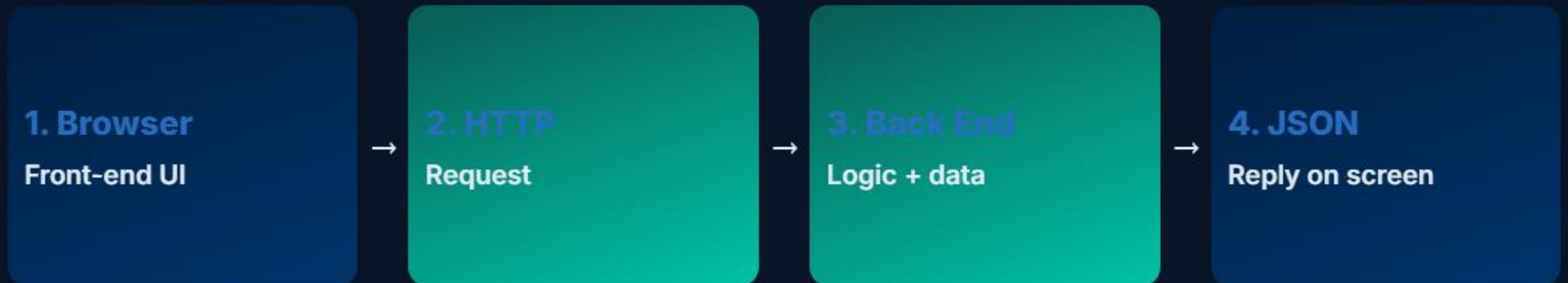
| Front End Vs Back End

- **Front end** — what people see and click in the browser
- **Back end** — the server that holds logic and data
- They talk over HTTP — the page sends a request, the server sends JSON back
- Why split them?
 - Secrets and heavy work stay on the server — the browser is public
 - You can change the look without rewriting the server (and the other way around)



| How A Web App Talks

- User types in the front-end page
- Browser calls a route on your back end
 - A **route** is a named URL path like `/api/chat` that runs one piece of server code
- Server runs the logic, data, and tools
- JSON comes back and the UI updates



JavaScript And Node.js

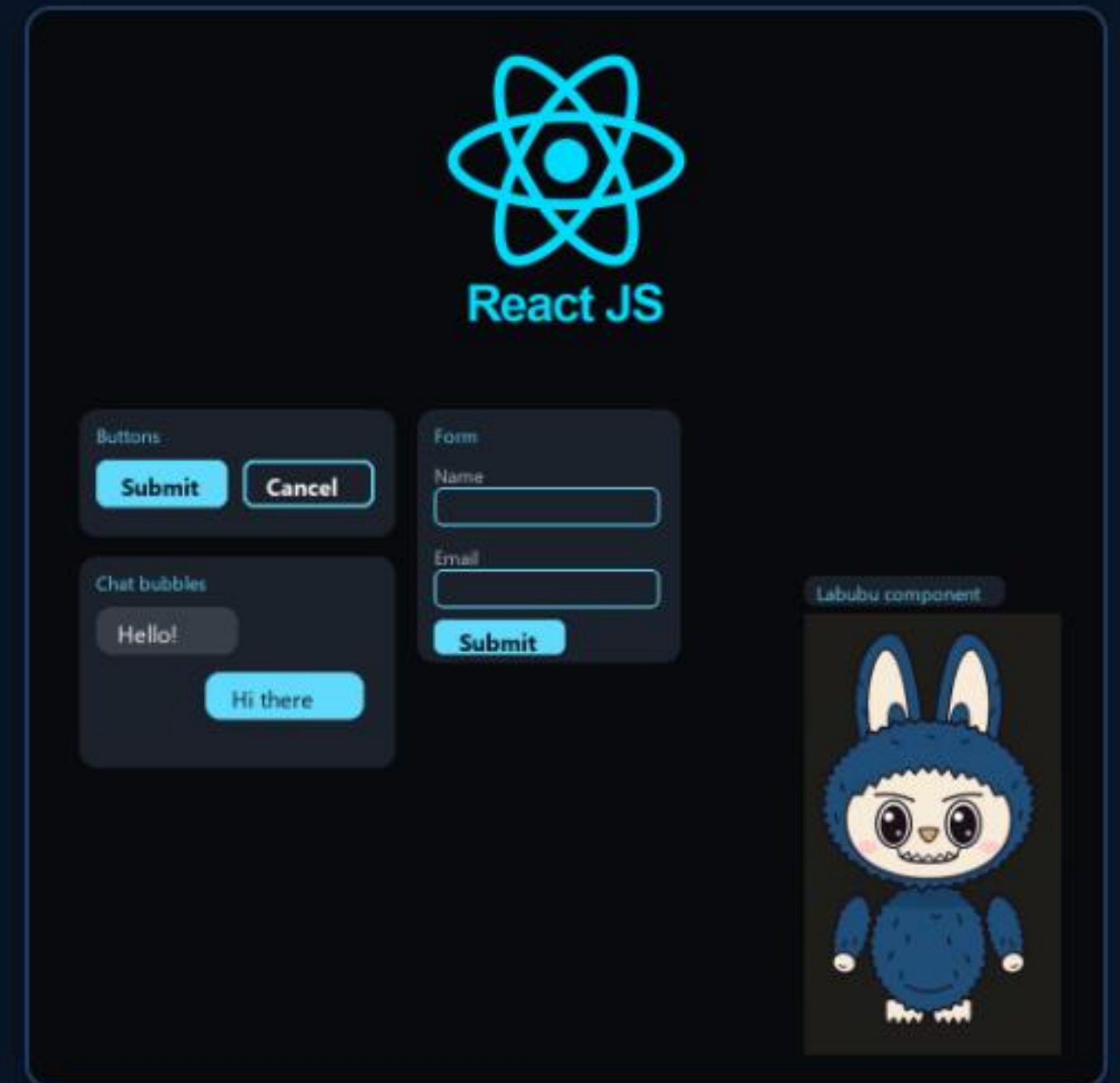
- Modern polished front ends are built in **JavaScript** — it's the language the browser already runs
- **Node.js** is the software that runs JavaScript on your laptop — like Python is the software that runs Python code
- **Important JavaScript commands** — **npm** means Node Package Manager; packages are listed in `package.json` (like Python's `requirements.txt`)
 - `npm install` — install all packages from `package.json` into `node_modules/`
 - `npm run dev` — run your app so you can open it in the browser



```
const msg = "hello";  
console.log(msg);
```

React

- **React** is a JavaScript library for building interactive web UIs from reusable pieces called **components**
- Built at **Facebook** (now Meta) to power the Facebook UI
- Called React because the page **reacts** when data changes — a new chat message shows up without reloading
- Popular for two main reasons
 - **Huge ecosystem** — Facebook, Instagram, Netflix, Airbnb, and many apps you use daily
 - **Reusable components** — a component is a UI piece you build once (a button, chat bubble, form, Labubu) and drop in anywhere



React Components

- A **component** is a function that returns a piece of UI — you call it like HTML: `<SignupForm />`
- **Props** (short for **properties**) are the inputs — change the props, change how it looks, without rewriting the design

Simple — a form

SignupForm.jsx

Copy

```
function SignupForm() {  
  return (  
    <form>  
      <input placeholder="Name" />  
      <input placeholder="Email" />  
      <button>Submit</button>  
    </form>  
  )  
}
```

SignupForm

Name

Email

Submit

Flexible — Labubu

Same SVG design; props change `fur`, `face`, `mood` (smile/eyes), `blush`

Labubu.jsx

Copy

```
<Labubu fur="#2f4a8a" face="#fbe9c9" mood="joy" blush={true} />  
  
function Labubu({  
  fur = "#e8778d",  
  face = "#fbe9c9",  
  mood = "classic",  
}) {  
  // ...  
}
```



JavaScript & TypeScript

- **JavaScript (JS)** — the language the browser runs for interactive pages
- **TypeScript (TS)** — JavaScript plus **types** (you label what a value should be: text, number, a user object, ...)
- Difference: JS lets almost anything through until it breaks in the browser; TS checks types while you write code and flags mismatches early
- Why many developers prefer TS — fewer “undefined is not a function” surprises, clearer code for teammates, better editor autocomplete; the browser still gets plain JS after a build step
- This course defaults to TypeScript with React for that safety net

JavaScript

.js

```
function greet(name) {  
  return "Hi " + name;  
}  
  
greet(42);  
// runs – then weird output later
```

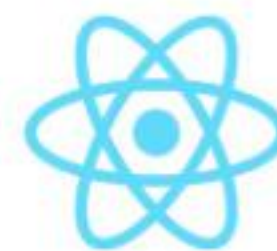
TypeScript

.ts

```
function greet(name: string) {  
  return "Hi " + name;  
}  
  
greet(42);  
// error: number is not a string
```

| Vite

- **Vite** (French for **fast**) is the tool that runs and builds your front-end project
- Different from React: React is the **UI library** (components on the page); Vite is the **dev server + bundler** that serves those files while you work
- Developers like it because it starts quickly, refreshes the page almost instantly when you save, and turns TypeScript/React into JavaScript the browser can run
- Common stack: **React + Vite + TypeScript**



React



Vite



TypeScript

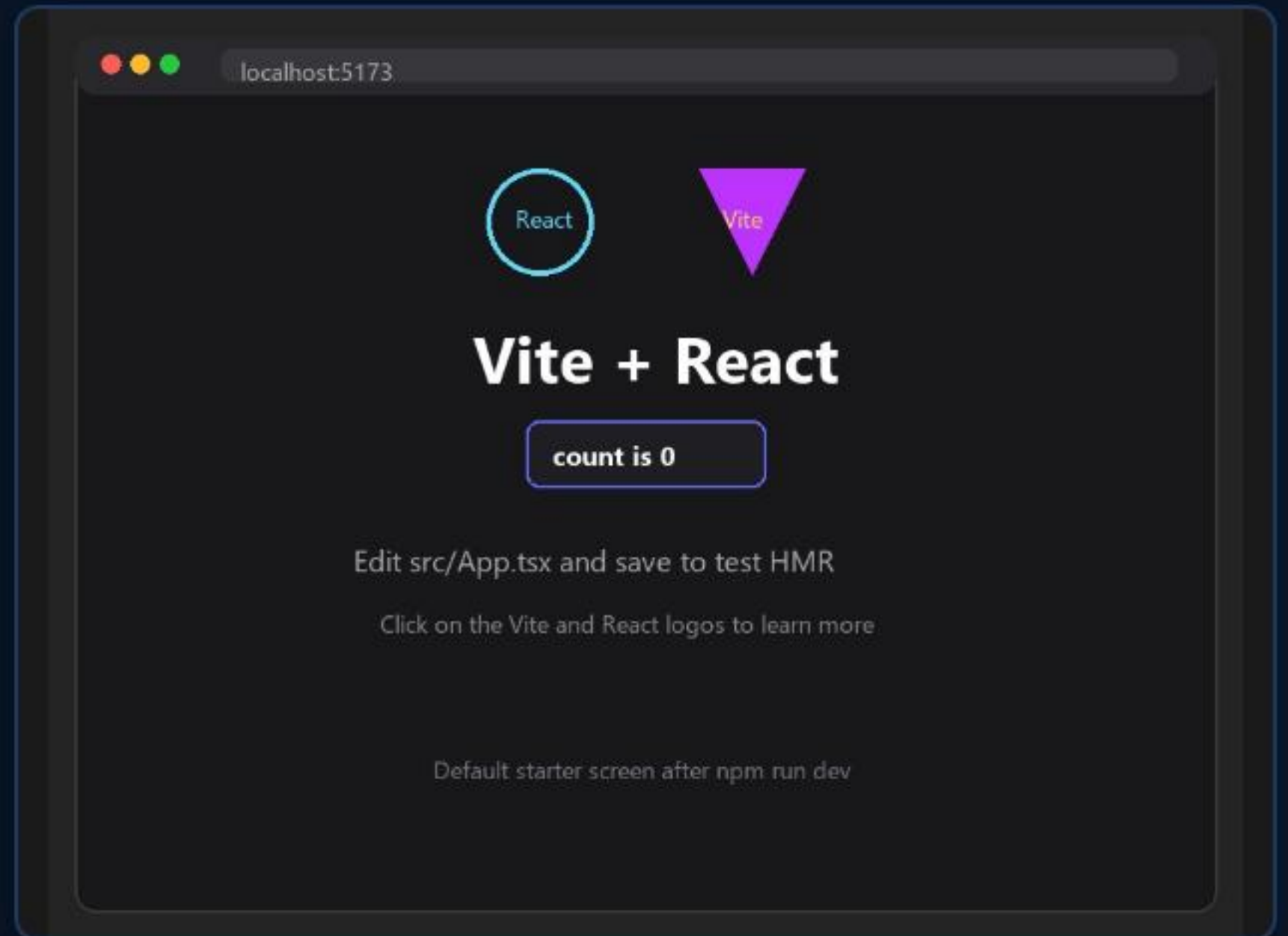
Scaffold A React Vite TS App

1. Scaffold into a `frontend/` folder — use `npm create vite@latest`
2. Change into that folder: `cd frontend`
3. Install modules: `npm install` → fills `node_modules/`
4. Run the app: `npm run dev` → you get the default Vite + React starter screen

terminal

Copy

```
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
npm run dev
```



| FastAPI Backend

- We prefer **Python** for the back end — you already know it, and our agents (PydanticAI + Portkey) live there
- **FastAPI** is a Python package for building web APIs — routes like `/api/chat` that return JSON
- Why it's nice
 - Fast to write — little boilerplate for endpoints
 - Plays well with **Pydantic** types and PydanticAI agents
 - Auto docs at `/docs` — you can test the back end without a running front end
 - Perfect partner for a React front end that fetches JSON

 **FastAPI**



Backend Layout & Run

- One project folder holds `frontend/` (React) and `backend/` (FastAPI + agent)
- **Uvicorn** is the server program that runs your FastAPI app and listens for HTTP requests (FastAPI is the code; Uvicorn is what keeps it online)
- We run the back end from `main.py` — in `backend/`, `uvicorn main:app` means “load the `app` object inside `main.py`”

```
Lecture7/  
  frontend/  
  backend/  
    main.py  
    agent.py  
    models.py  
    tools.py  
    prompts/  
      prompt.md
```

terminal

Copy

```
cd backend  
uvicorn main:app --port 8000
```

API Routes & `main.py`

- `main.py` has the API routes for the back end
- A **route** is one URL path + one function — FastAPI wires them with decorators like `@app.get` / `@app.post`
- Similar to how tools are created in PydanticAI — decorate a function, give it a job

backend/main.py (routes)

Copy

```
from fastapi import FastAPI
from agent import run_agent

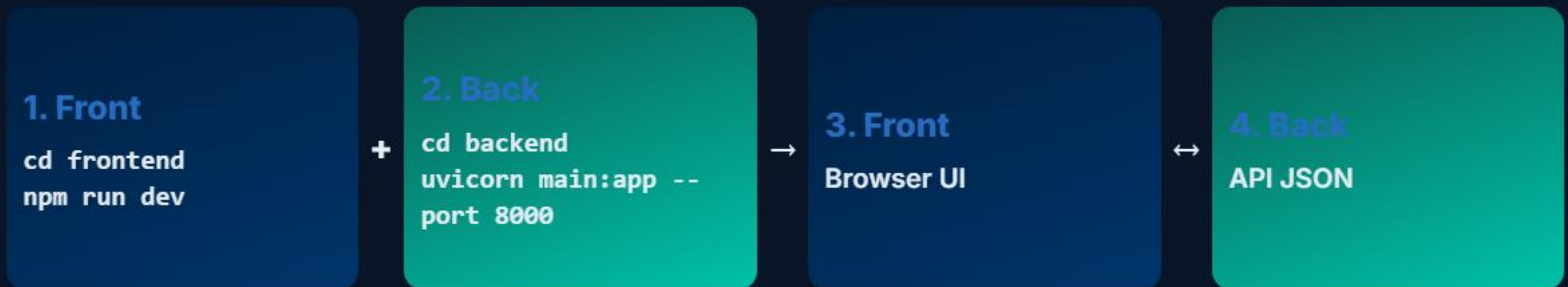
app = FastAPI()

@app.get("/api/health")
def health():
    return {"ok": True}

@app.post("/api/chat")
def chat(body: dict):
    result = run_agent(body["message"])
    return {
```

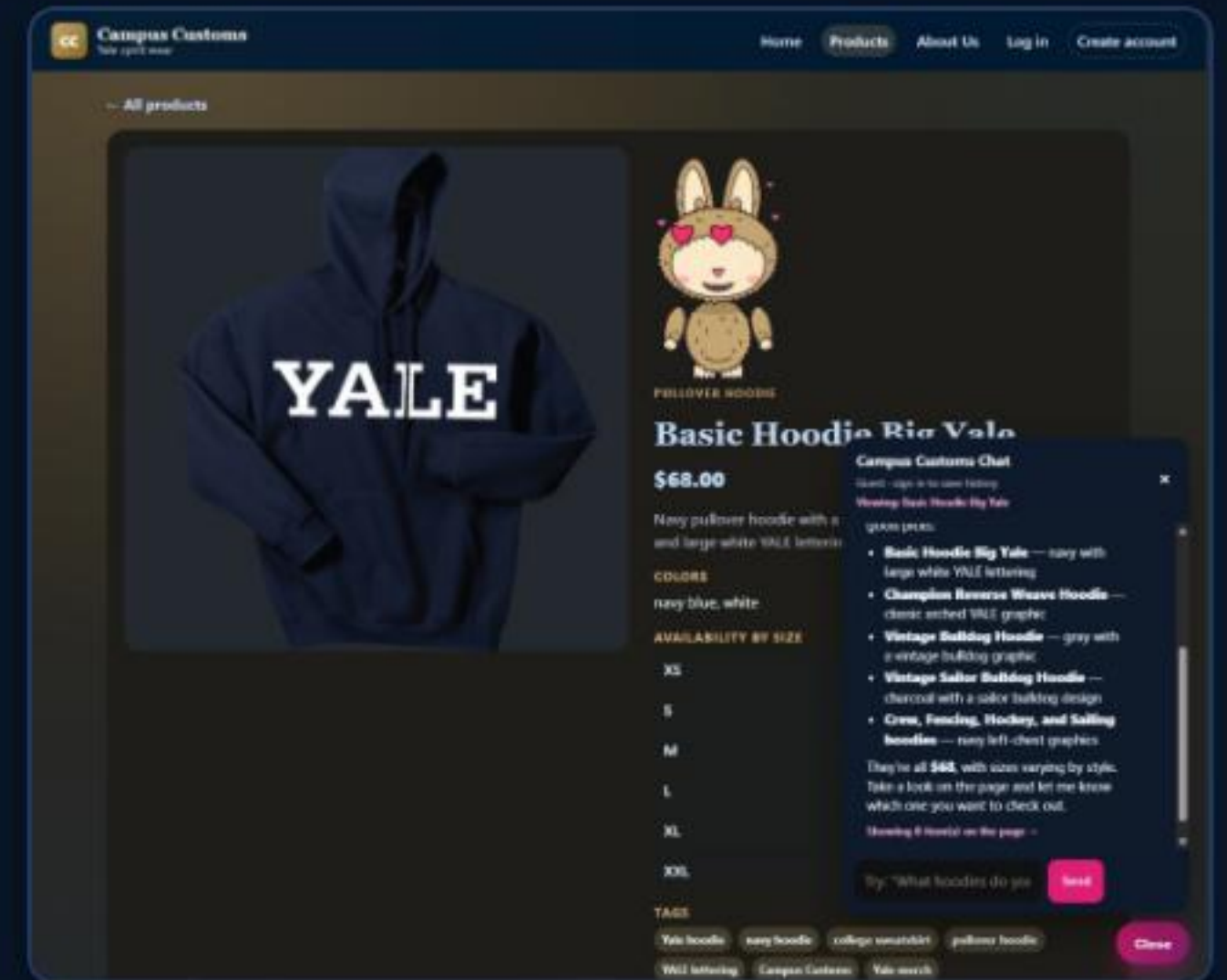
Two Processes At Once

- Our apps are **front + back** — two programs running at the same time
- We run the **front** with Vite: `npm run dev` (browser UI)
- We run the **back** with Uvicorn: `uvicorn main:app --port 8000` (API + agent)
- **CORS** (Cross-Origin Resource Sharing)
 - By default the browser **blocks** your React page (port 5173) from reading answers from the API (port 8000) — different origins
 - FastAPI CORS middleware **allows** it — the API response basically says “this front end is allowed to use me”



Full Stack Agent Chatbot App

- Pretty chat front end — **React + Vite + TypeScript**
- Back end — **FastAPI** with a **PydanticAI** agent
- Optional data sources — `data.json` or a database



Build Full Stack Chat Agent

- JSON data file
- Web search tools
- Make the chat window look pretty
- Vibe coding session →

